

Architecture Specification

Program topology, account model, PDA conventions, and the instruction surface of RoundFi

RoundFi
2026-06-23

CONTENTS

1. Program topology
2. Account and PDA model
3. The instruction surface
4. The pause carve-out
5. Validated topology

Derived document. This is a derivation of [docs/spec/MASTER-SPEC.md](#) (§4 Protocol → instructions, §11 Deployed state). The Master Spec is the single source of truth; if an ID or schema here disagrees with it, the Master Spec wins. Every program ID, account, and instruction is pinned to the deployed Jun 2026 source (`programs/` , `crates/math/`). Citations point at the source file so this document can be re-verified, not trusted.

1. Program topology

RoundFi is **four Solana programs**. One holds the protocol; one holds the portable reputation; two are interchangeable yield adapters behind a common interface. Keeping reputation in its own program is what makes the score **portable** — it is not entangled with any single pool's lifecycle, and an external consumer can read it without touching pool state.

Program	Role
<code>roundfi-core</code>	All protocol logic — pool lifecycle, the four vaults, contributions, default settlement, the Escape Valve, the pause circuit breaker.
<code>roundfi-reputation</code>	The on-chain level ladder and attestation records — score, tier, completed-pools, the identity gate.
<code>roundfi-yield-mock</code>	A devnet yield adapter (deterministic, for end-to-end testing of the Yield Cascade).
<code>roundfi-yield-kamino</code>	The mainnet-target yield adapter — the Kamino CPI integration.

1.1 Deployed program IDs

The four programs are deployed on devnet and **share their IDs across clusters** (MASTER-SPEC §11):

Program	ID
roundfi-core	8LVrgxKwKwqjcdq7rUUwWY2zPNk8anpo2JsaR9jTQQjw
roundfi-reputation	Hpo174C6JTCfiZ6r8VYVQdKxo3LBHaJmMbkgREkxe9R2
roundfi-yield-mock	GPTMPgxexhwkhXNovnfrcSsmoWPUhedvKAQfTV2Ef5AQ
roundfi-yield-kamino	74izMa4WzLuHvtzDLdNzcygKe5fYwtD95EiWMuzhFdb

1.2 The yield-adapter interface

`roundfi-core` does not call Kamino directly. The two yield programs are **pluggable adapters** behind one interface: `deposit_idle_to_yield` and `harvest_yield` issue CPIs to whichever adapter the pool is configured against. `roundfi-yield-mock` is the devnet implementation; `roundfi-yield-kamino` is the mainnet target (on-chain CPI shipped, operational reserve pin pending). Swapping adapters does not touch pool or reputation state — the float simply routes through `yield_vault` to a different program.

2. Account and PDA model

2.1 The Pool account

The **Pool** PDA is the spine of a deployment. It is allocated by `create_pool` and carries the life-cycle status (`Forming` / `Active` / `Completed` / `Closed`), the geometry (`members_target` , `cycles_total` , `current_cycle`), the timing (`next_cycle_at` , `cycle_duration`), the split parameters (`escrow_release_bps`), and the Yield-Cascade earmarks (`guarantee_fund_balance` , `lp_distribution_balance`). A core invariant is fixed at allocation: `cycles_total == members_target` (`create_pool.rs:123` , SEV-038) — every slot is drawn exactly once.

2.2 The Member account

A **Member** PDA records one wallet's position in one pool: `contributions_paid` , `escrow_balance` , `on_time_count` , `late_count` , `slot_index` , the stake tier, and the `defaulted` flag. This is **per-pool operational state**, distinct from the wallet's portable reputation (§2.5). On an Escape Valve sale the seller's Member PDA is **closed** and the buyer's is **created** carrying the **position-state snapshot** — `contributions_paid` , `escrow_balance` , `on_time_count` , `late_count` , `slot_index` , stake tier — so obligations transfer with the slot while reputation does not.

2.3 The four vaults

Each pool owns **four USDC token accounts**, created by `init_pool_vaults` , all with **PDA authorities**:

Vault	Purpose
<code>pool_usdc_vault</code>	The pool float — contributions land here net of splits; payouts and seized funds flow here.
<code>escrow_vault</code>	Member stakes + the escrow slice of each contribution; the vesting / seizure surface.
<code>solidarity_vault</code>	The " Cofre Solidário " 1%-of-installment reserve; first tranche of any default.
<code>yield_vault</code>	Idle USDC parked into the configured yield adapter.

Vault creation is split out of `create_pool` into `init_pool_vaults` because allocating the Pool PDA and all four ATAs in a single instruction would exceed Solana's **stack-depth** budget.

2.4 The seizure / vesting surface

`escrow_vault` and `solidarity_vault`, together with the member stake, are the physical accounts the **Triple Shield** acts on. `settle_default` seizes from them in a fixed order (solidarity → escrow → stake); `release_escrow(checkpoint)` vests stake back out of escrow for an on-time member. The mechanics are specified in [02-technical-whitepaper](#) §4–§5; here they are listed as the accounts those instructions mutate.

2.5 The reputation accounts (`roundfi-reputation`)

The portable credit signal lives in its own program, in its own PDAs:

- **ReputationProfile** — keyed `[b"reputation", wallet]`. Holds the wallet's **score**, **tier** (L1–L4), `cycles_completed`, and identity flag. Because it is keyed by **wallet**, it is **non-transferable** and tracks the wallet across every pool — an Escape Valve buyer keeps their own profile and inherits none of the seller's. This is the account an external lender reads to price credit.
- **IdentityGateConfig** — the configurable identity gate. Field `required_min_level` governs whether L2/L3 require verification (devnet `0` = open, mainnet `3` = verified-only). The L4 hard floor (`IDENTITY_HARD_FLOOR_LEVEL`) is **independent** of this config — Elite **always** requires `identity_verified`, even with the gate off.

The four-tier ladder, the score schedule, and the Pass-3 attestation schemas that write these accounts are specified in [04-behavioral-reputation-score](#).

2.6 Position NFTs

Each pool slot is represented by a **position NFT** (mpl-core). On an Escape Valve buy the NFT is **thawed** → **transferred** → **re-frozen** via mpl-core CPIs, authorized by the slot's `position_authority` PDA. The frozen-by-default state is what binds the NFT to its slot's obligations between trades.

2.7 The contemplation accounts

Two PDAs decide **who receives the carta, and when** (MASTER-SPEC §4.8):

- **DrawResult** — keyed `[b"draw-result", pool]`, written once by `finalize_draw` when a sorteio pool fills. It stores `order[seat] = cycle`, which is a **permutation** — and that single fact is what makes "every member is contemplated exactly once" a property of the data structure rather than an invariant anyone has to enforce. The seed is `draw-result` and not `draw` precisely because the shorter stem reads as `SEED_DRAW_BPS`, the unrelated 91.6% seed-draw constant.
- **Bid** — keyed `[b"bid", pool, cycle, bidder]`, one sealed envelope per bidder per cycle. Holds `commit_hash: [u8; 32]`, the revealed `amount` and `parcels`, and a `Committed → Revealed` state. Putting the **cycle in the seeds** means a commitment can never be replayed into a different cycle, and a second commit for the same cycle collides on `init` instead of quietly re-sealing after a peek.

`Pool` also carries two single-byte fields for this subsystem — `ordering_policy` and `current_bid_depth` — both **carved from trailing padding** so `Pool::SIZE` never changed and pools created before the feature existed decode fine, reading a zeroed byte as the safe default.

3. The instruction surface

The instruction surface, grouped by lifecycle phase. Every named instruction below appears in MASTER-SPEC §4; nothing is invented here.

3.1 Pool creation and membership

Instruction	Effect
<code>create_pool</code>	Allocates the Pool PDA; enforces <code>cycles_total == members_target</code> and the Seed Draw viability guard (reverts <code>PoolNotViable</code>).
<code>init_pool_vaults</code>	Creates the four USDC vault ATAs (split from <code>create_pool</code> for stack depth).
<code>join_pool</code>	Member joins and locks stake; flips the pool Active when <code>members_joined == members_target</code> .

3.2 The contribution cycle

Instruction	Effect
<code>contribute</code>	Pays an installment; splits it 1% / 25% / 74% into solidarity / escrow / float; emits the on-time, late, or pool-complete attestation.
<code>claim_payout</code>	The drawn member claims the carta; advances <code>current_cycle</code> , re-arms <code>next_cycle_at</code> ; flips Completed on the final cycle.
<code>re-lease_escrow</code>	Vests stake back out of escrow for an on-time member, gated on <code>on_time_count ≥ checkpoint</code> (else <code>EscrowLocked</code>).

3.3 Contemplation — sorteio and lance

Who gets the carta, and in what order. Specified in MASTER-SPEC §4.8; decided in [ADR 0012](#).

Instruction	Effect
<code>finalize_draw</code>	Permissionless. When <code>pool.ordering_policy == 1</code> (sorteio), draws the contemplation order on-chain once the pool fills and freezes it into a <code>DrawResult</code> PDA.
<code>place_embedded_bid</code>	Bids the installments the member has already prepaid . Depth is <code>contributions_paid - current_cycle - 1</code> ; strictly greater than <code>pool.current_bid_depth</code> wins.
<code>place_bid_commit</code>	Seals a free bid: writes <code>sha256(amount salt bidder)</code> into a Bid PDA. Requires <code>clock < pool.next_cycle_at</code> . Uses <code>init</code> , not <code>init_if_needed</code> — re-sealing is what a commitment must forbid.
<code>place_bid_reveal</code>	Opens the envelope during <code>next_cycle_at ≤ clock < next_cycle_at + GRACE_PERIOD_SECS</code> . Adjudicates before transferring , so a losing bid reverts with the USDC still in the bidder's wallet.

Three properties carry this subsystem:

- **DrawResult.order[seat] = cycle is a permutation**, and a winning bid **swaps** two of its entries. A transposition of a permutation is still a permutation, so "every member is contemplated exactly once" holds **by construction** — and `claim_payout / crank_payout` required no changes to support bidding at all.
- **The windows are disjoint**. Commits close exactly where reveals open, so a bid can never be placed in response to one already revealed. That is the whole anti-snipe mechanism; it needs no extra state.
- **Losing is free apart from the fee**, because eligibility is checked before any token moves. This is why there is no bid vault, no refund path, and no settlement instruction — the free bid reduces to prepayment plus an embedded bid, and a K-installment bid settles with **one** attestation (`BehavioralPayload.parcels_paid`).

3.4 Default settlement

Instruction	Effect
<code>settle_default</code>	Seizes solidarity → escrow → stake into <code>pool_usdc_vault</code> ; marks <code>defaulted</code> ; emits <code>SCHEMA_DEFAULT</code> . Eligibility gated on the 7-day mainnet / 1-day canary grace window.

3.5 Yield Cascade

Instruction	Effect
<code>depos-it_idle_to_yield</code>	Routes idle <code>pool_usdc_vault</code> float into <code>yield_vault</code> via the adapter CPI.
<code>harvest_yield</code>	Realizes yield and runs the 20% fee → guarantee fund → 65% LP → residual waterfall; <code>min_realized</code> slippage floor.

3.6 Escape Valve (secondary market)

Instruction	Effect
<code>escape_valve_list</code>	Direct path: lists a position at a clear <code>price</code> , immediately <code>Active</code> .
<code>escape_valve_list_commit</code>	Commit-reveal path: posts <code>SHA-256(price salt)</code> , hiding the price.
<code>escape_valve_list_reveal</code>	Reveals <code>(price, salt)</code> ; arms <code>buyable_after = now + REVEAL_COOLDOWN_SECS</code> (30s).
<code>escape_valve_buy</code>	Atomically transfers price, swaps the Member PDA (position-state snapshot), moves the NFT, closes the listing; enforces <code>now ≥ buyable_after</code> (<code>ListingNotBuyableYet</code>).

3.7 Termination and rent reclaim

Instruction	Effect
<code>close_pool</code>	Pure terminal <code>Completed</code> → <code>Closed</code> ; decrements committed TVL, moves no funds .
<code>close_member</code>	Reclaims a Member PDA's rent to the member (the rent-reclaim ceremony, SEV-039).
<code>close_pool_vaults</code>	Sweeps residual USDC → treasury, closes the vault ATAs + Pool PDA, returns rent → authority.

`close_pool` only flips the status; the **true** end of a pool's life is the rent-reclaim ceremony — `close_member` × N (rent → members) followed by `close_pool_vaults` (residual → treasury, accounts closed, rent → authority).

3.8 Reputation and identity (`roundfi-reputation`)

Instruction	Effect
<code>promote_level</code>	Permissionless : advances a wallet to the highest tier whose score and completed-pools thresholds are both met (<code>resolve_level</code>), then applies <code>cap_level_for_identity</code> .

4. The pause carve-out

`pause(bool)` (authority-gated) sets `config.paused`. The gate is implemented as a `!config.paused` **account-validation constraint** on the shared `config` account, so paused instructions revert with `ProtocolPaused` **before** any handler logic runs.

17 instructions are gated while paused — `create_pool`, `join_pool`, `contribute`, `claim_payout`, `crank_payout`, `release_escrow`, `skip_defaulted_payout`, `deposit_idle_to_yield`, `harvest_yield`, **all four Escape Valve paths**, `cancel_pending_listing`, and **all three lance paths** (`place_embedded_bid`, `place_bid_commit`, `place_bid_reveal`).

`settle_default` is the deliberate carve-out. A default already in flight must remain settleable while the rest of the protocol is frozen, so that a pause cannot trap a pool with an unrecoverable delinquent member. Structurally this means `settle_default` does **not** carry the `!config.paused` constraint that the gated instructions do — the exemption is enforced by the **absence** of the gate.

`finalize_draw` is outside the gate for a different reason: it takes no `config` account at all, so it is structurally un-pausable rather than deliberately exempted. It is permissionless and moves no funds — it only freezes a `DrawResult` for a pool that has filled — and every instruction that would *pay* against that order is itself gated. The distinction matters: `settle_default` is an exemption someone chose, while this one is a consequence of the account list. Recorded so it is reviewed rather than discovered.

5. Validated topology

Seven of the eight capability areas were exercised **end-to-end on devnet** on 2026-06-12 with real USDC; the eighth (`settle_default`) is armed and time-gated. The instruction surface above is therefore not aspirational — the lifecycle (`Ga2RwgSk...`), the Yield Cascade and both Escape Valve paths (`4SZCKeQL...`), the pause gate, escrow vesting, and the full rent-reclaim ceremony have all run on the deployed programs. Transaction signatures and the reproducible runbook live in `docs/operations/v52-devnet-runbook.md`.

Cross-references: the end-to-end protocol mechanics these accounts and instructions implement → [02-technical-whitepaper](#); the reputation engine, ladder, and Pass-3 schemas behind `roundfi-reputation` → [04-behavioral-reputation-score](#). The canonical instruction list and program IDs live in MASTER-SPEC §4 and §11.